

Sugestao IA

Essa percepção é muito comum em projetos de pesquisa e sistemas emulados: começamos a cobrir todos os *corner cases* possíveis (expansão de templates de nós, amostradores periódicos de kernel em threads, checagens exaustivas de schema) e o código se torna um monstro de 3.000 linhas antes mesmo do fluxo principal estar validado ponta a ponta.

Para a **v1 (MVP)**, o objetivo deve ser um só: **carregar a topologia, levantar a rede WiFi/Containers, rodar o tcpdump, disparar os binários (receiver e sender), e salvar o PCAP, o CSV com os tempos e o summary.json.**

Aqui está o desenho completo dessa versão enxuta para "abrir a mente":

1. O que sai da frente agora (para depois)

- **✗ Remover:** Amostrador contínuo de contadores de interface em thread de background (`CounterSampler`).
 - **✗ Remover:** Snapshots complexos de tabelas internas do BATMAN-adv (`batctl tg` , `batctl tl`).
 - **✗ Remover:** Sistema genérico de templates e expansão de nós (`node_groups` , herança de chaves). Cada nó ou grupo é declarado diretamente.
 - **✗ Remover:** Dezenas de políticas de fallback (`missing_binaries: idle/skip/fail`). Se faltar o binário, simplesmente aborta com erro claro.
-

2. Estrutura de Pastas Final (Simples e Limpa)

Mantendo a **Opção A** (tudo centralizado e sem conflito de `sudo`):

Demo-1-Testbed/

└─ bin/

│ └─ receiver

Binário compilado do

protocolo

│ └─ sender

Binário compilado do

protocolo

└─ dockerfiles/

│ └─ Dockerfile.drone

└─ logs/

Destino das métricas

geradas

│ └─ traffic.pcap

│ └─ metrics.csv

│ └─ summary.json

└─ scripts/

│ └─ config.yml

Arquivo de configuração

enxuto

│ └─ run.py

Ponto de entrada CLI

(apenas chama o testbed)

│ └─ testbed/

```
|      |— __init__.py
|      |— config.py          # Lê o config.yml e aplica
defaults simples
|      |— network.py        # Cria nós Containernet,
WiFi e interfaces IP/bat0
|      |— runner.py         # Sobe receiver, espera
READY e dispara sender
|      |— metrics.py        # Inicia tcpdump, extrai
tempos e gera CSV + summary
```

3. O que cada arquivo vai fazer

scripts/run.py (~50 a 80 linhas)

- Recebe o caminho do YAML via terminal (ex: `sudo python3 run.py config.yml`).
- Chama em sequência:
 1. `cfg = load_config("config.yml")`

2. `net = build_network(cfg)`
3. `dump_proc = start_tcpdump(cfg)`
4. `results = execute_protocol(cfg, net)`
5. `stop_tcpdump(dump_proc)`
6. `generate_reports(cfg, results)`

- No bloco `finally:`, garante que a rede encerre e executa `mn -c`.

testbed/config.py (~60 a 90 linhas)

- Faz o `yaml.safe_load`.
- Valida apenas o essencial: se os nós declarados possuem posições válidas, se o modo é `unicast` ou `broadcast` e se a porta foi informada.
- Preenche valores padrão (ex: potência do sinal, canal WiFi) caso não estejam no YAML.

testbed/network.py (~150 a 200 linhas)

- Cria o `Containernet` com `wmediumd` (interferência).
- Adiciona as estações (`DockerSta`) para cada Drone e GCS especificados no config.
- Configura as interfaces sem fio (`wlan0` em ad-hoc com o BSSID/SSID).
- Sobe a interface `bat0` (BATMAN-adv) e atribui o IP definido para cada container.

`testbed/runner.py` (~150 a 200 linhas)

- **Receiver:** Executa dentro do container designado:

```
./receiver --address <IP> --port <PORTA>
```

Lê a saída e aguarda a linha `READY` .
- **Sender:** Assim que os receivers derem `READY` , dispara:

```
./sender --mode <unicast|broadcast> --destination  
<IP_DESTINO> --port <PORTA> --count <N>
```
- Aguarda a finalização do sender e captura o tempo total de resposta, códigos de saída e eventuais erros.

testbed/metrics.py (~100 a 150 linhas)

- **tcpdump**: Inicia a captura em background na interface do cenário (ex: `bat0` ou `wlan0`), gravando em `logs/traffic.pcap`.
- **Cálculo de Tempos (`metrics.csv`)**: Lê os tempos de envio/resposta registrados pelo protocolo ou extraídos via timestamp de pacotes e calcula:
 - Média, Mediana, Mínimo, Máximo e Desvio Padrão.
 - Grava o resultado em `logs/metrics.csv`.
- **summary.json**: Salva o status de execução de cada nó:
 - Status (`success` ou `failed`), exit code, tempo de execução e total de pacotes enviados/recebidos.

4. O `config.yml` Inteiro (Versão Limpa)

Aqui está um modelo de configuração direto ao ponto, contemplando tanto **Unicast** quanto **Broadcast**:

```
#
=====

=====
# Configuração do Experimento
#
=====

=====
experiment:
  id: "exp-01-unicast"
  duration_seconds: 60

#
=====

=====
# Parâmetros de Comunicação e Modo
#
=====

=====
communication:
```

```
# Opções: "unicast" ou "broadcast"
```

```
mode: "unicast"
```

```
port: 5000
```

```
packets_count: 100
```

```
# Configuração dos papéis:
```

```
# No modo Unicast: sender -> container específico, receiver -  
> container específico.
```

```
# No modo Broadcast: sender -> container específico,  
receivers -> lista de containers.
```

```
sender: "drone1"
```

```
receivers:
```

```
- "gcs"
```

```
#
```

```
=====
```

```
=====
```

```
# Parâmetros de Rede Sem Fio (WiFi + BATMAN-adv)
```

```
#
```

```
=====
=====
wireless:
```

```
    ssid: "meshNet"
```

```
    channel: 5
```

```
    bssid: "02:11:22:33:44:55"
```

```
    # Subnet da rede BATMAN-adv
```

```
    subnet: "192.168.123.0/24"
```

```
    # IP de destino em broadcast (usado automaticamente quando
mode == "broadcast")
```

```
    broadcast_ip: "192.168.123.255"
```

```
#
```

```
=====
=====
# Definição dos Nós da Topologia
```

```
#
```

nodes:

Estação de Controle Terrestre (GCS)

- name: "gcs"

type: "gcs"

ip: "192.168.123.1/24"

position: [0, 0, 0] *# [x, y, z] em metros*

image: "drone:latest"

Drones

- name: "drone1"

type: "drone"

ip: "192.168.123.2/24"

position: [15, 20, 10]

image: "drone:latest"

- name: "drone2"

type: "drone"

ip: "192.168.123.3/24"

position: [30, 40, 10]

```
image: "drone:latest"

#
=====

=====
# Saída e Métricas
#
=====

=====
output:
  logs_dir: "logs"
  pcap_filename: "traffic.pcap"
  metrics_filename: "metrics.csv"
  summary_filename: "summary.json"
```

5. Como funciona a lógica entre Unicast e Broadcast

A orquestração fica trivial no código:

1. **Se** `mode: "unicast"` :

- Pega o IP do receiver indicado em `receivers[0]` .
- Comando do Sender:

```
./sender --mode unicast --destination 192.168.123.1 --port 5000 --count 100
```

2. **Se** `mode: "broadcast"` :

- Sobe o `./receiver` em **todos** os nós listados em `receivers` .
- Aguarda o `READY` de todos eles.
- O destino do sender passa a ser o IP de broadcast configurado (`192.168.123.255`):

```
./sender --mode broadcast --destination 192.168.123.255 --port 5000 --count 100
```

Resumo dos Benefícios Dessa Simplificação

- **De 3.170 linhas para ~600-700 linhas no total** somando todos os arquivos.
 - **Foco total nos requisitos:** Você tem os binários rodando, o socket abrindo, tráfego capturado no `.pcap`, métricas resumidas no `.csv` e o status de cada nó no `.json`.
 - **Fácil de debugar:** Se algo der errado na rede, você mexe apenas em `network.py`; se o protocolo travar, mexe em `runner.py`.
 - **Pronto para crescer:** Quando esse fluxo estiver 100% testado e gerando dados para a sua pesquisa, você adiciona novas métricas ou mobilidade sem refatorar o núcleo.
-

Duvidas

- oq eh count? precisamos? podemos simplificar?
- nao precisa esperar mensagem de `READY` do receiver, so executar sem erro e ta bom

- sender/receiver precisa receber o `mode` ? acho q n
- nao precisa no config.yml os param de output, ja temos um lugar fixo.
- nomes dos arquivos em testbed/ precisam ser mais auto-explicativos